CS 380: ARTIFICIAL INTELLIGENCE

Lecture 3: Problem Solving & Uninformed Search

Problem Solving

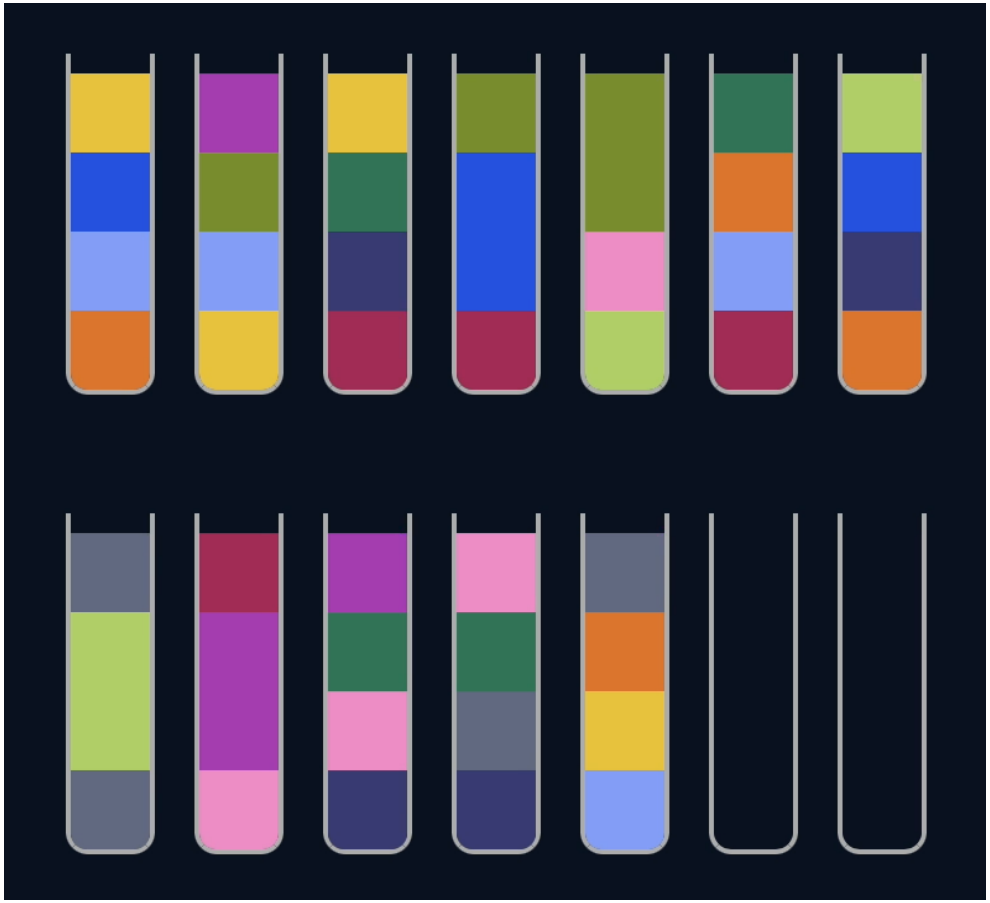
- What is a “problem”?
  - In the context of AI: a question the requires an answer
- Many types of problems
  - **Decision Problems**
  - **Search Problems**
  - **Counting Problems**
  - **Optimization Problems**

Problem Solving

- There are many problems for which we know specialized ways of solving.
- For example:
  - Problem: Find the roots of the polynomial $ax^2 + bx + c = 0$
  - We have mechanical procedures to solve this problem in an exact way
  - However, those procedures can only be applied to this problem, not any other
- Problem Solving in AI:
  - Finding general procedures to solve general classes of problems
  - Are there algorithms that can be used to solve any computationally solvable problem?
    - We humans seem to be able to!

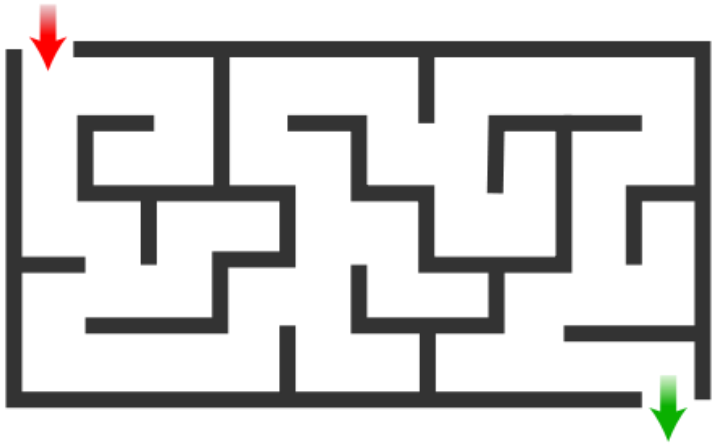
Problem Solving

- Example: Beakers / Water Jugs Puzzle
  - Pour from one to another but only on top of same color (or empty)
  - Continue until beakers are filled with only 1 color



Example: Maze

- A robot is at the entrance of the maze.
- Problem: How does the robot get to the exit?
- Step 1: Formulate goal
  - Be in the exit
- Step 2: Formulate problem
  - States, actions, etc.
- Step 3: Find solution
  - Specific sequence of actions



Problem Formulation

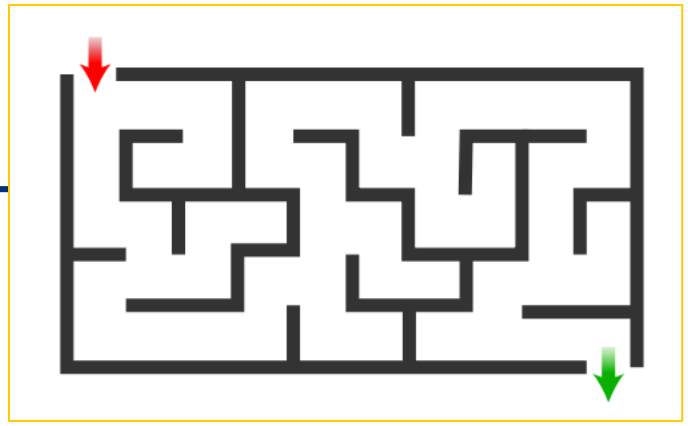
- Initial state: S_0
  - Initial configuration of the problem (e.g. starting position in a maze)
- Actions: ***A***
  - The different ways in which the agent can change the state (e.g. moving to an adjacent position in the maze)
- Goal condition: ***G***
  - A function that determines whether a state reached by a given sequence of actions is a solution to the problem or not
- Cost function: ***c***
  - A function that assigns a numeric cost to a given sequence of actions

Problem Formulation

- Initial state: S_0
  - Initial configuratio
    maze)
- Actions: ***A***
  - The different ways
    (e.g. moving to an
- Goal condition: ***G***
  - A function that determines whether a state reached by a given sequence of actions is a solution to the problem or not
- Cost function: ***c***
  - A function that assigns a numeric cost to a given sequence of actions

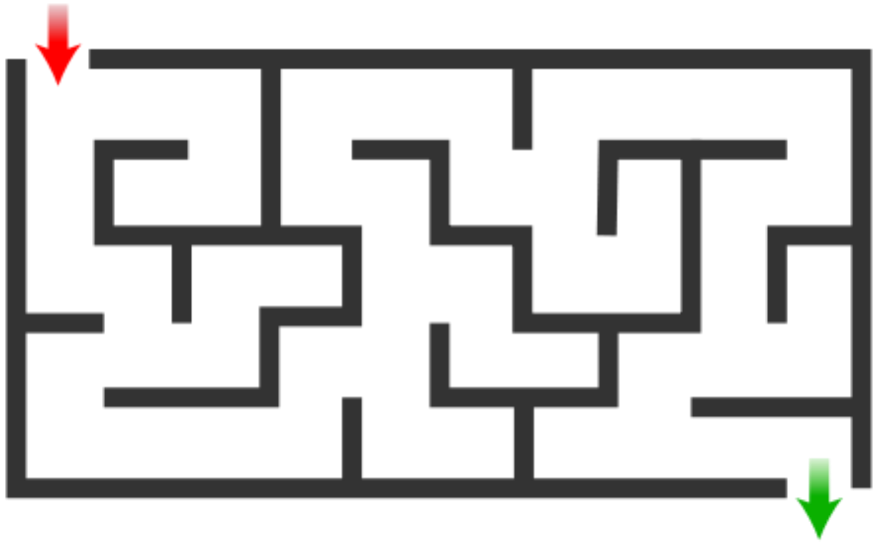
The initial state, together with the set of actions, define the State Space or Search Space — the set of all the possible states that are reachable from the initial state by some sequence of actions.

Problem Types



- Deterministic, fully observable:
  - Solution: [down, down, down, right, down, …]
- Non-deterministic or partially observable:
  - Agent might not know its exact coordinates, must rely on percepts to tell them about the walls around them
  - Solution, policy (mappings from percepts, or states, to actions):
    - If no wall on left, then turn left
    - If wall on left, no wall ahead, then advance
    - If wall on left, wall ahead, then turn right
  - For example: the “follow the right/left wall” strategy

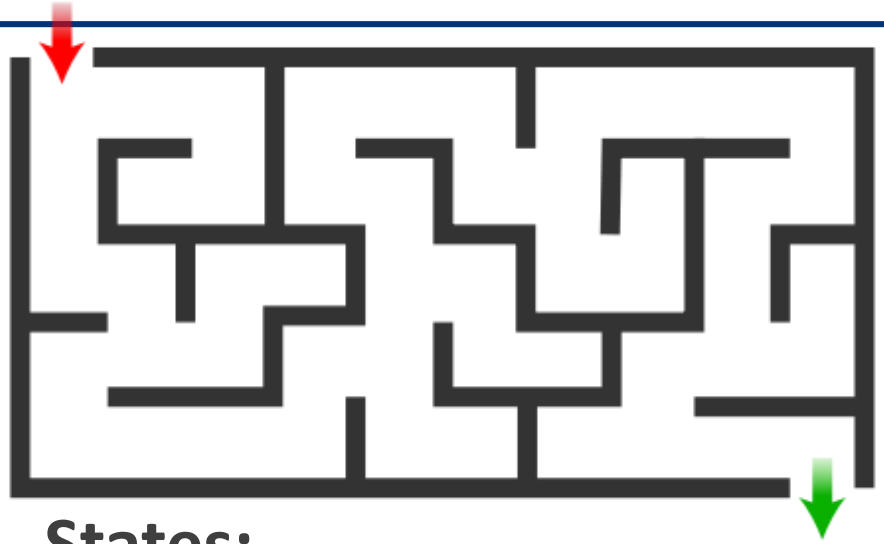
Example: Maze



- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

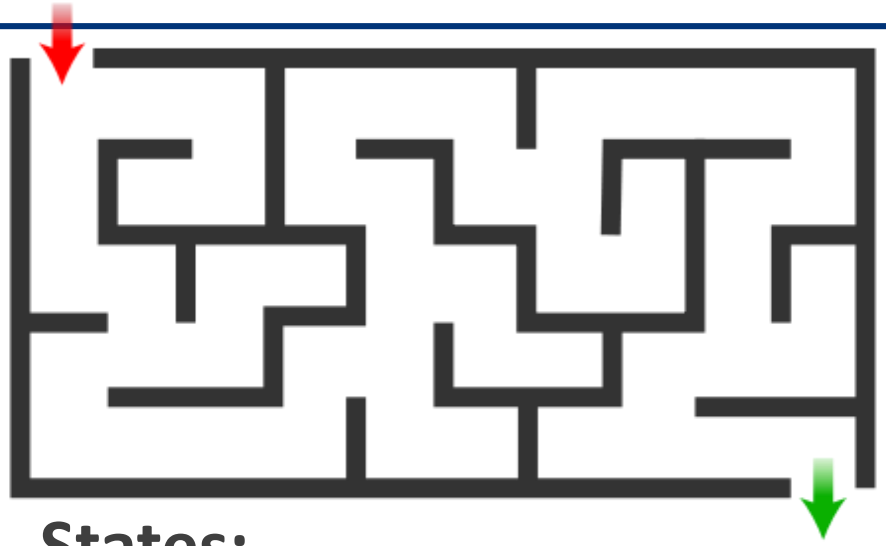
Assuming we have a robot with a direction and partial information…

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

Example: Maze

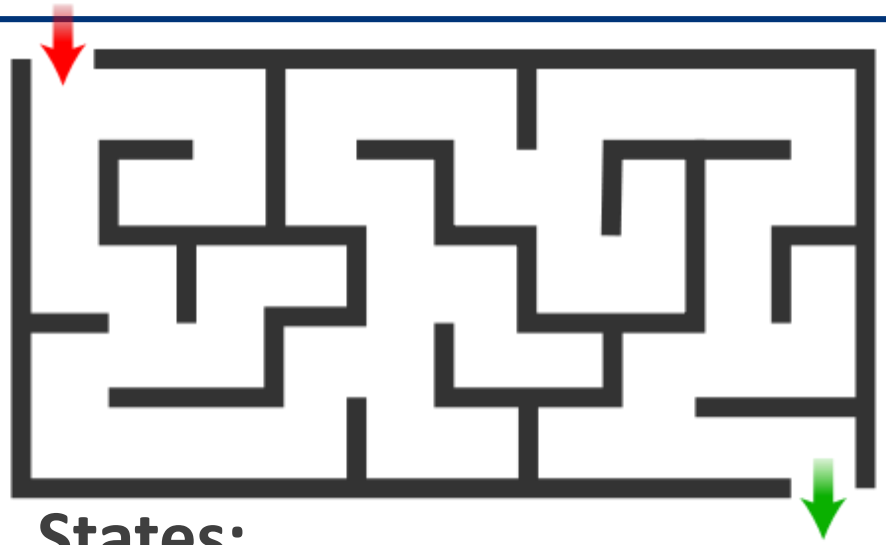


- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

How difficult is this problem?

Let's say, compared to a larger maze…
or chess…

Example: Maze



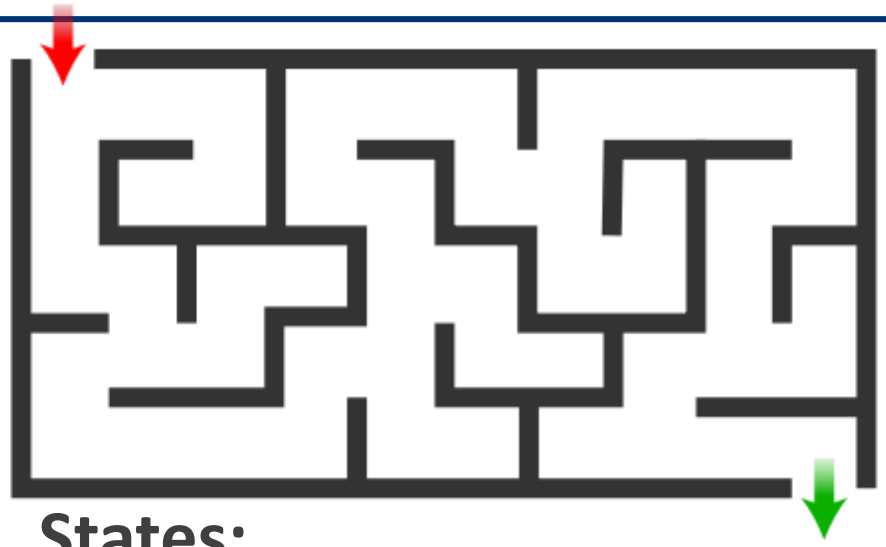
- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

How many possible states does this maze have? (for one agent finding its way around…)

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

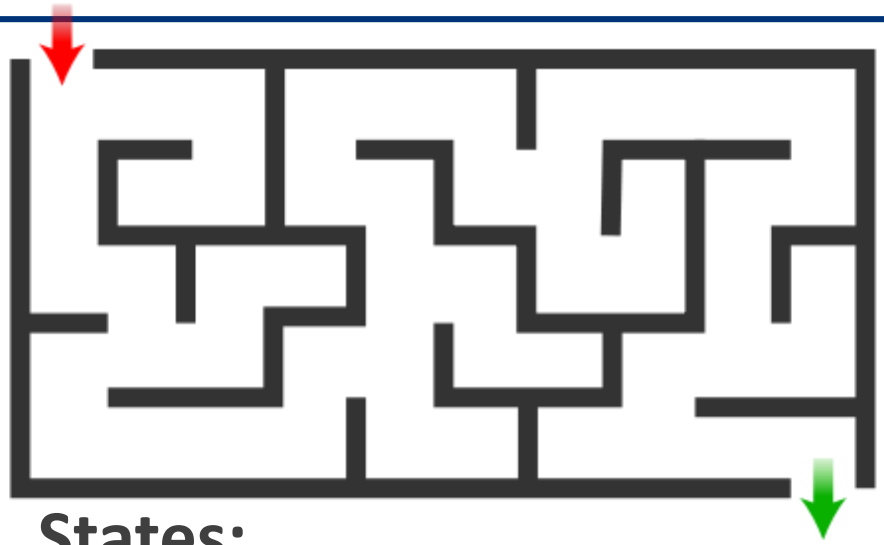
How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

For this maze, the state space has 200 states (50 positions * 4 directions)

How about chess?

Example: Maze



- **States:**
  - ((x,y), facing direction)
  - Initial state: ((0,0), down)
- **Actions:** advance, left, right
- **Goal:** reach state ((9,4), *)
- **Cost:** 1 per action executed

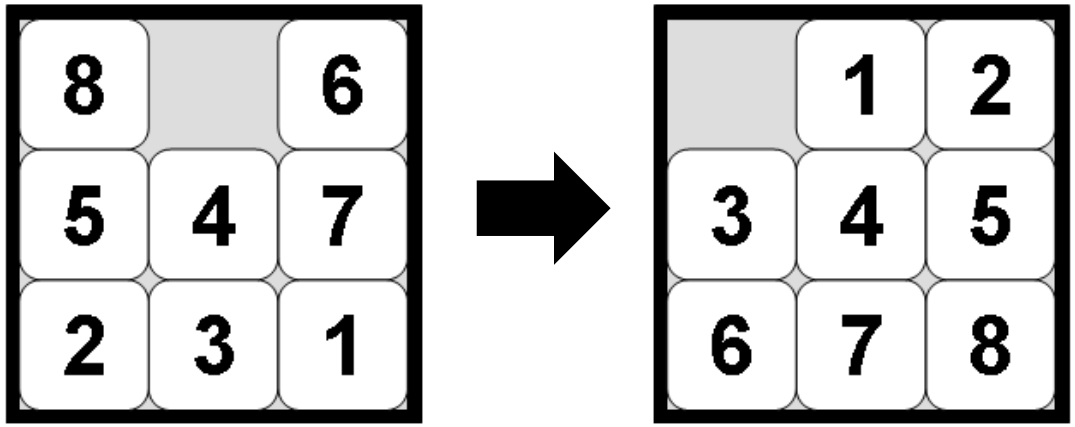
How difficult is this problem?

A typical measure of "difficulty" is the size of the state space: if it's small, it's very easy to find the solution.

For this maze, the state space has 200 states (50 positions * 4 directions)

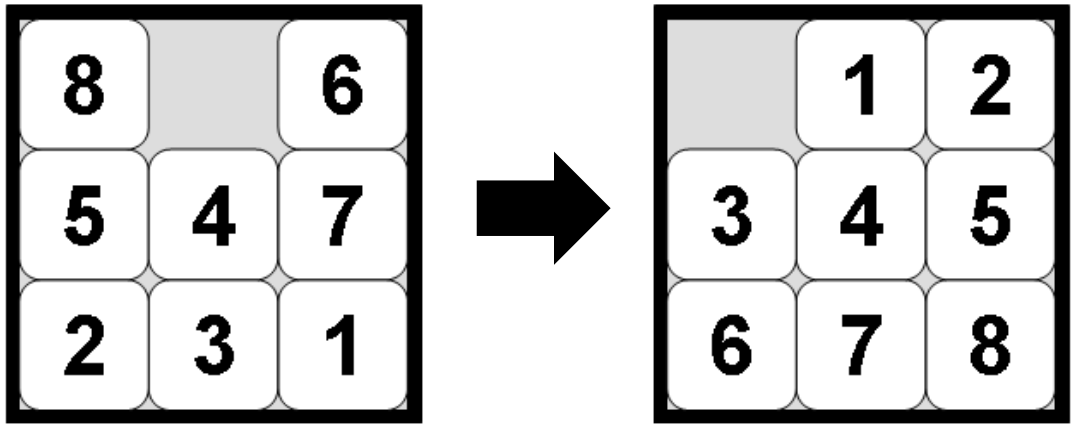
Chess has about 10^{50} states

Example: 8-puzzle



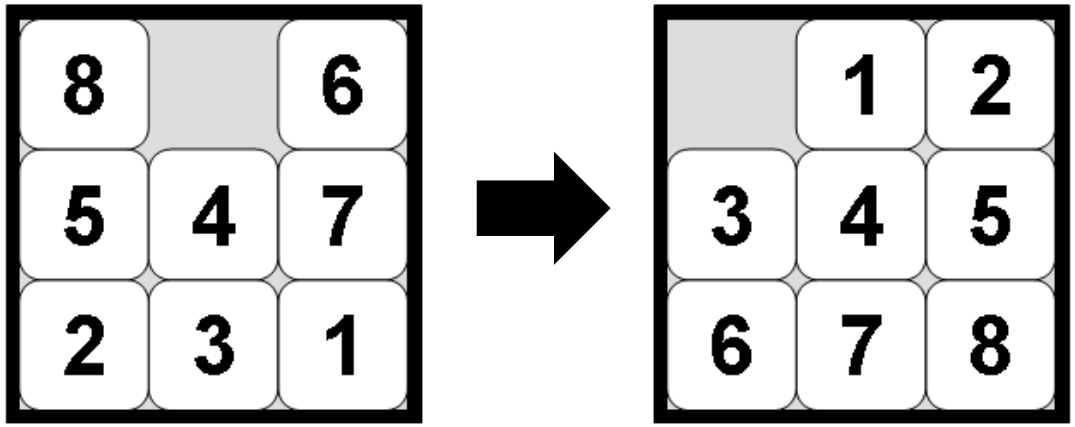
- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

Example: 8-puzzle



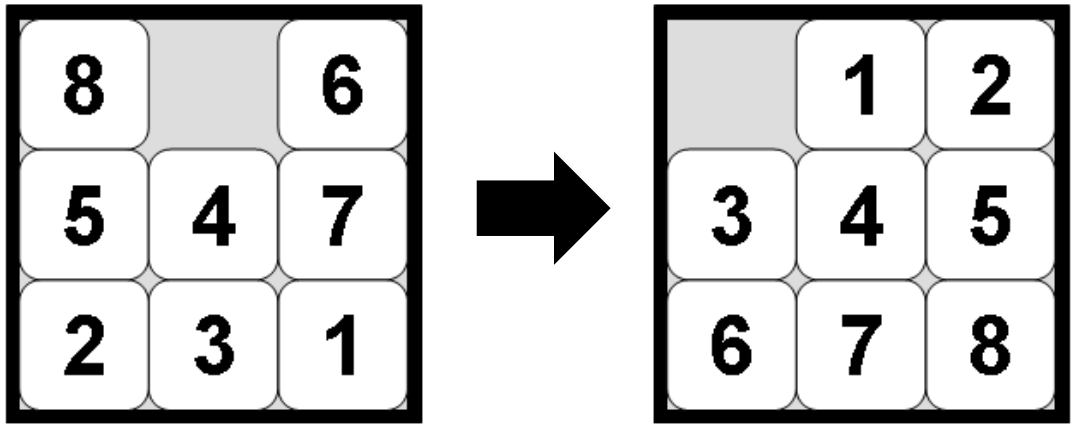
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:**
- **Goal:**
- **Cost:**

Example: 8-puzzle



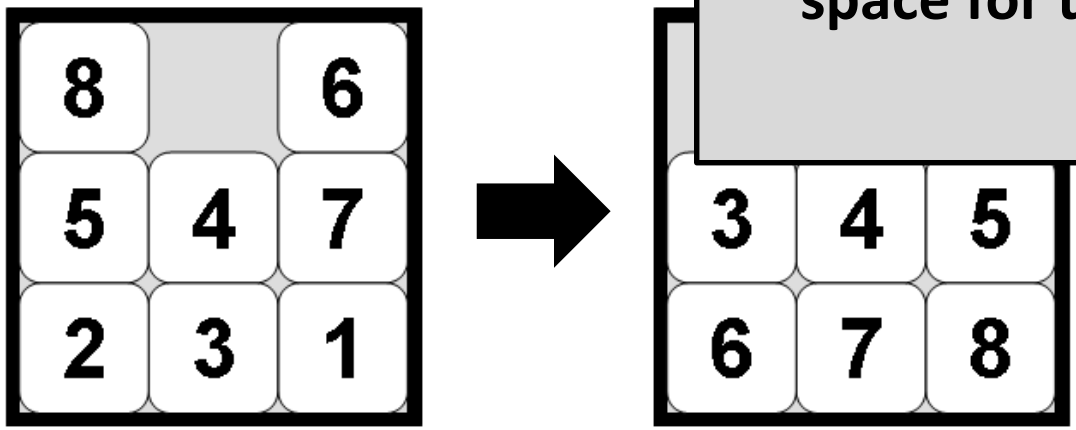
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:**
- **Cost:**

Example: 8-puzzle



- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

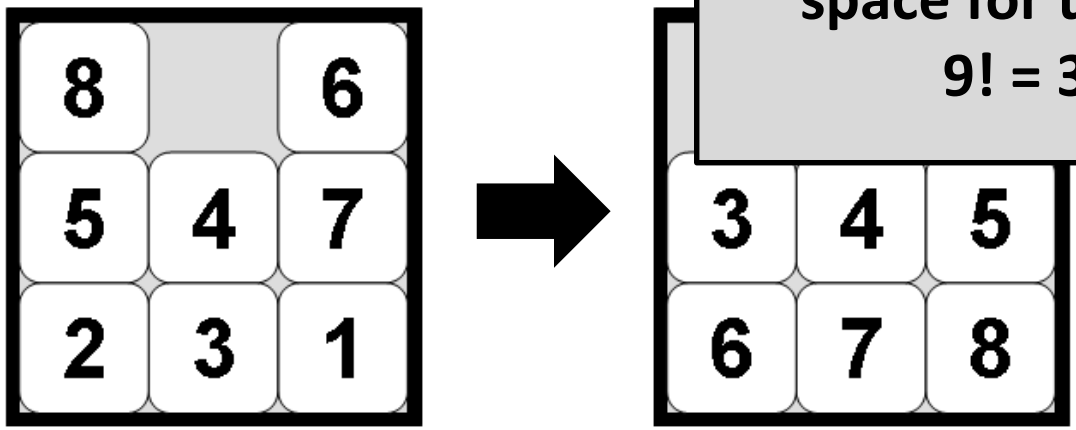
Example: 8-puzzle



What's the size of the state space for the 8-puzzle?

- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

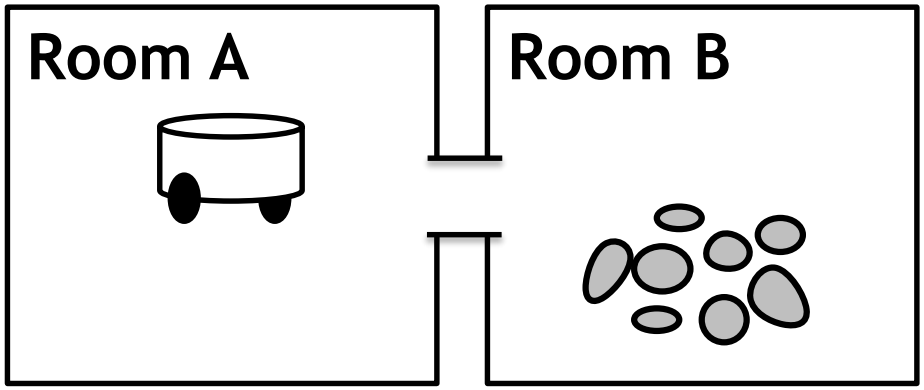
Example: 8-puzzle



What's the size of the state space for the 8-puzzle?
9! = 362,880

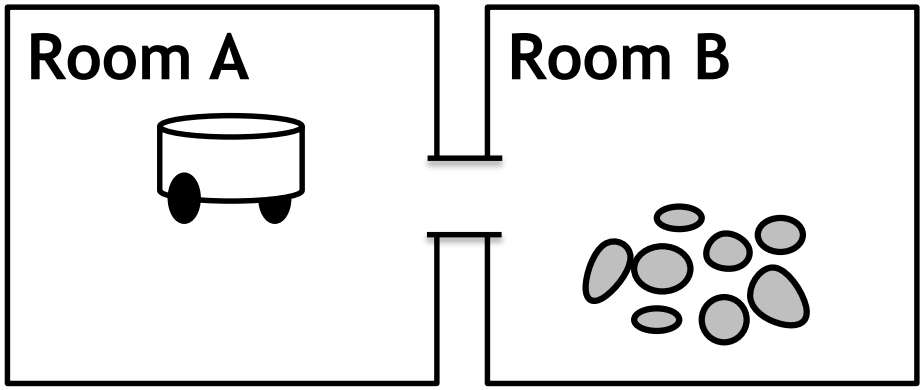
- **States:**
  - Board configuration, represented as a vector (1d or 2d) of 9 positions, where each position represents one cell on the board
  - Initial state: <8,blank,6,5,4,7,2,3,1>
- **Actions:** "move" the blank up, down, left, right
- **Goal:** Reach state <blank,1,2,3,4,5,6,7,8>
- **Cost:** 1 per move

Example: Vacuum World



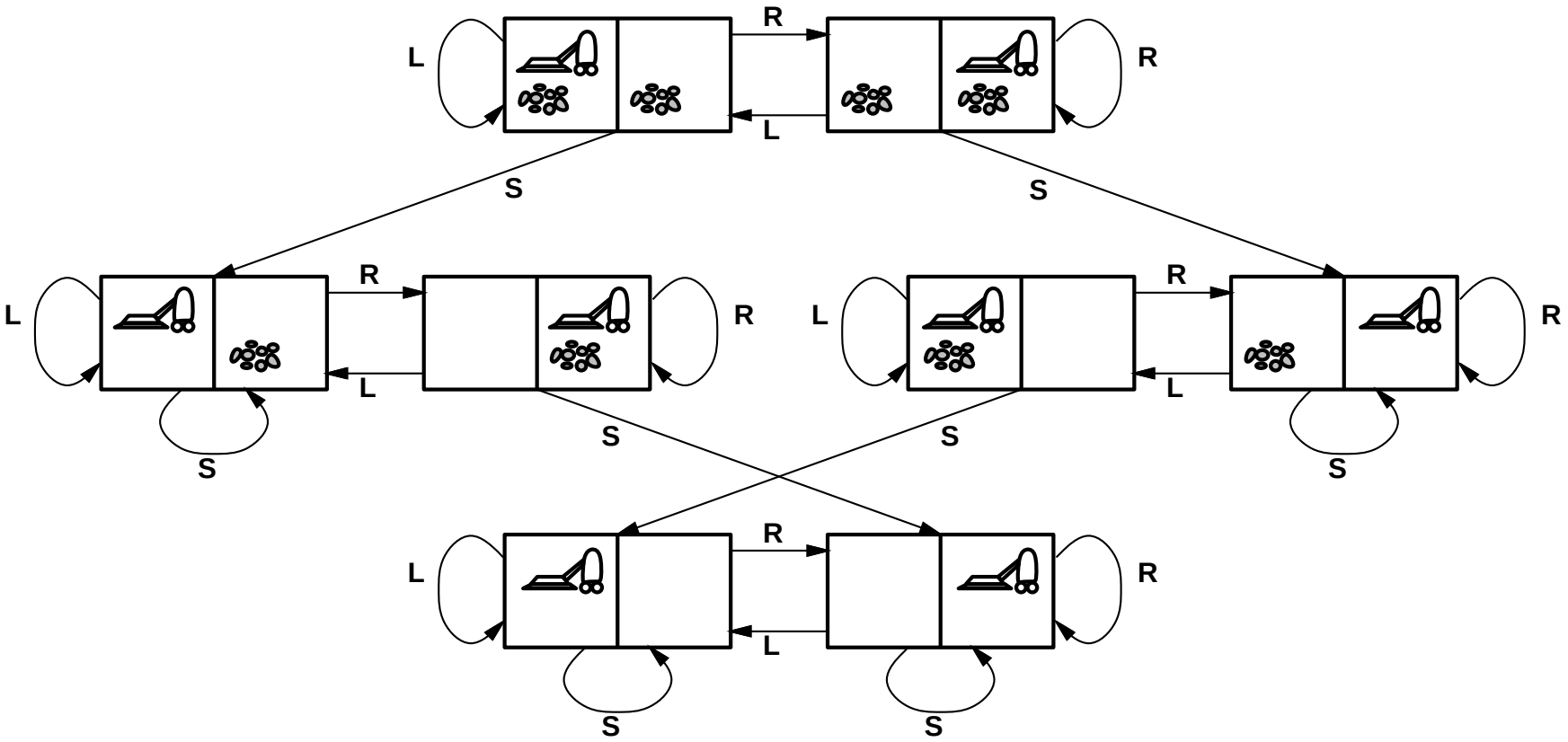
- States:
- Actions:
- Goal:
- Cost:

Example: Vacuum World

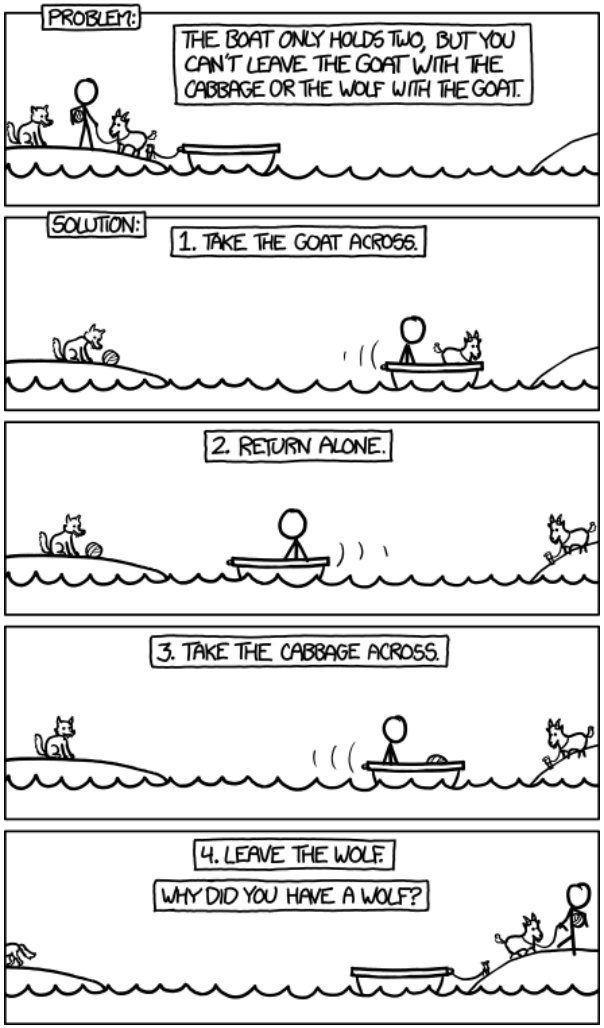


- **States:**
  - Position of robot + dirt in A? + dirt in B?
  - Initial state: (A, no, yes)
- **Actions:** Left, right, vacuum
- **Goal:** (-, no, no)
- **Cost:** 1 per move, 1 per vacuum (??)

Example: Vacuum World

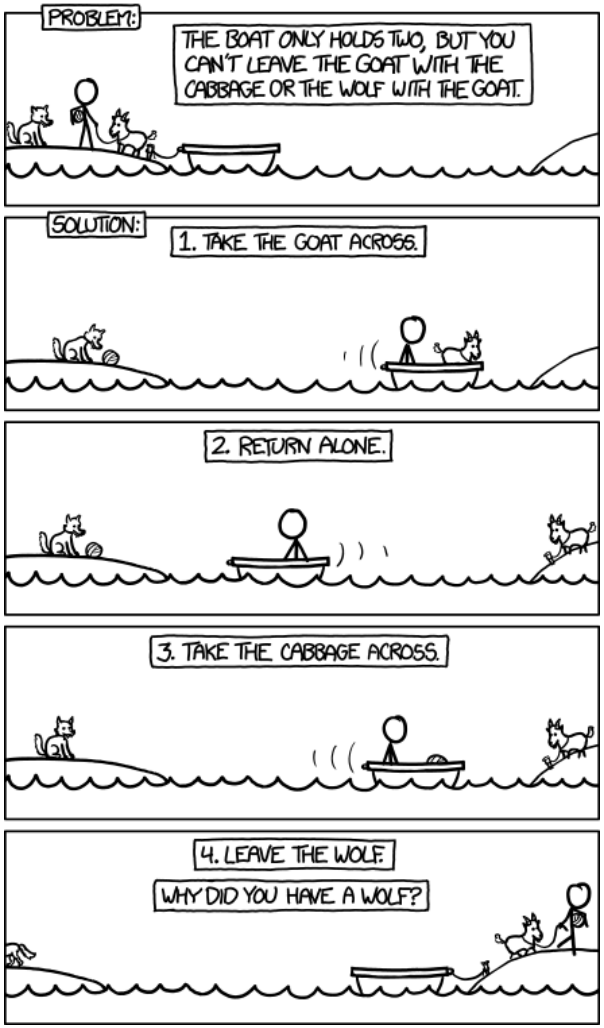


Example: Wolf, Goat, Cabbage



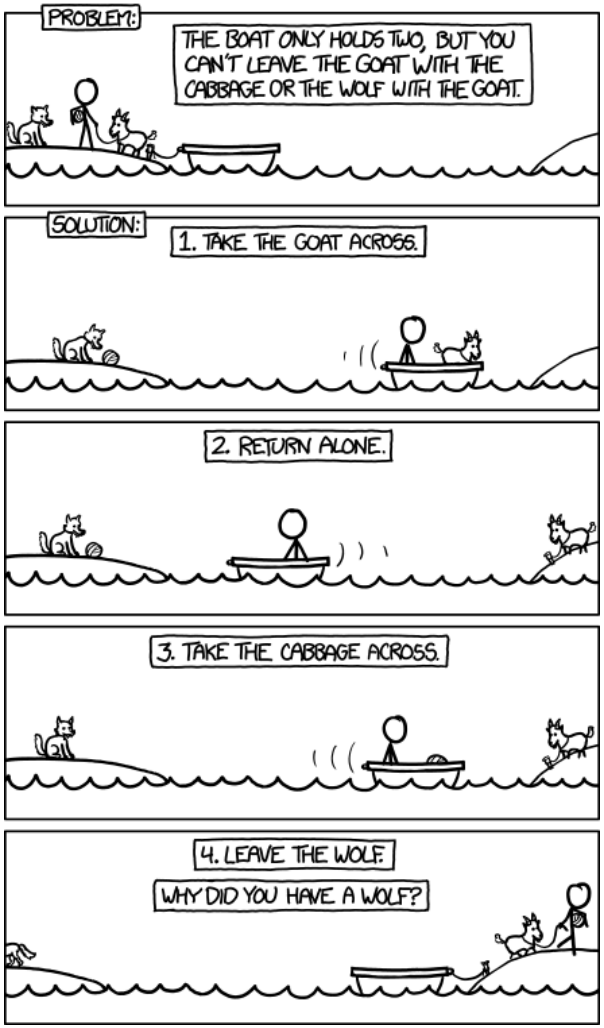
- **States:**
- **Actions:**
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



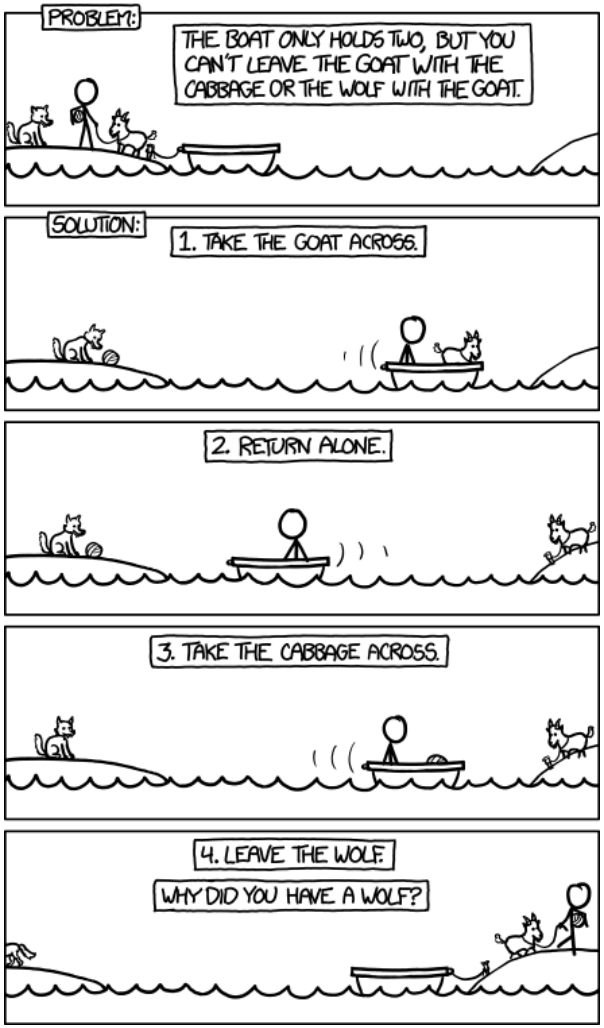
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



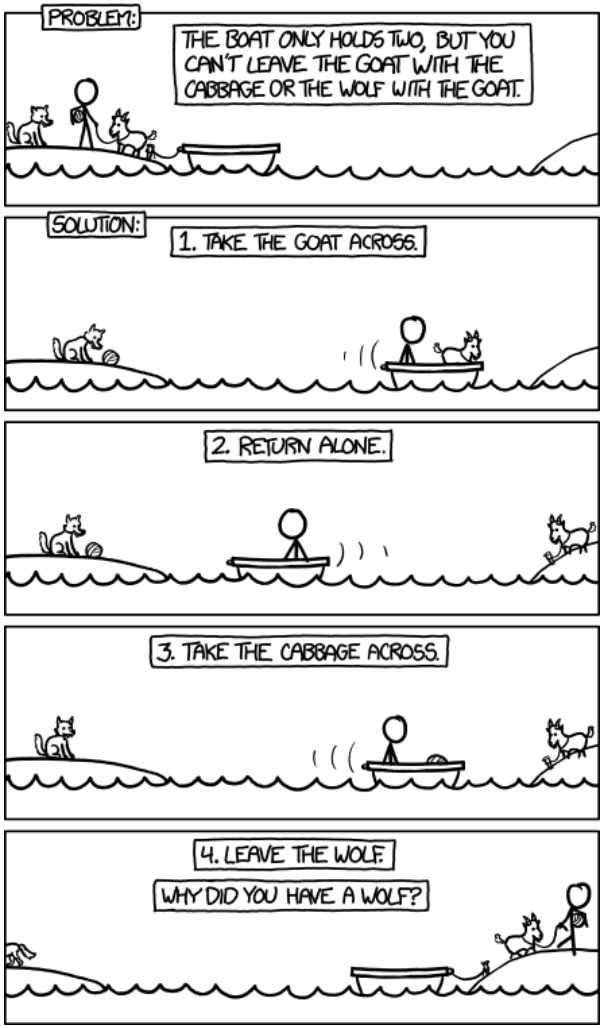
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:**
- **Cost:**

Example: Wolf, Goat, Cabbage



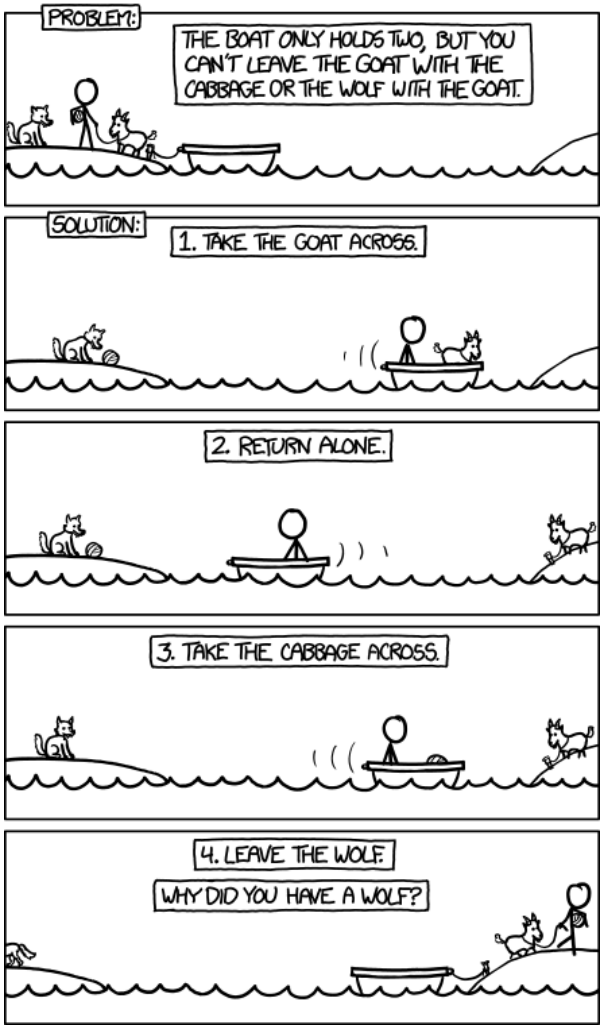
- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:**

Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial state: (left, left, left, left)
- **Actions:**
  - Boat left, boat right
  - Load wolf, unload wolf
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:** 1 per action

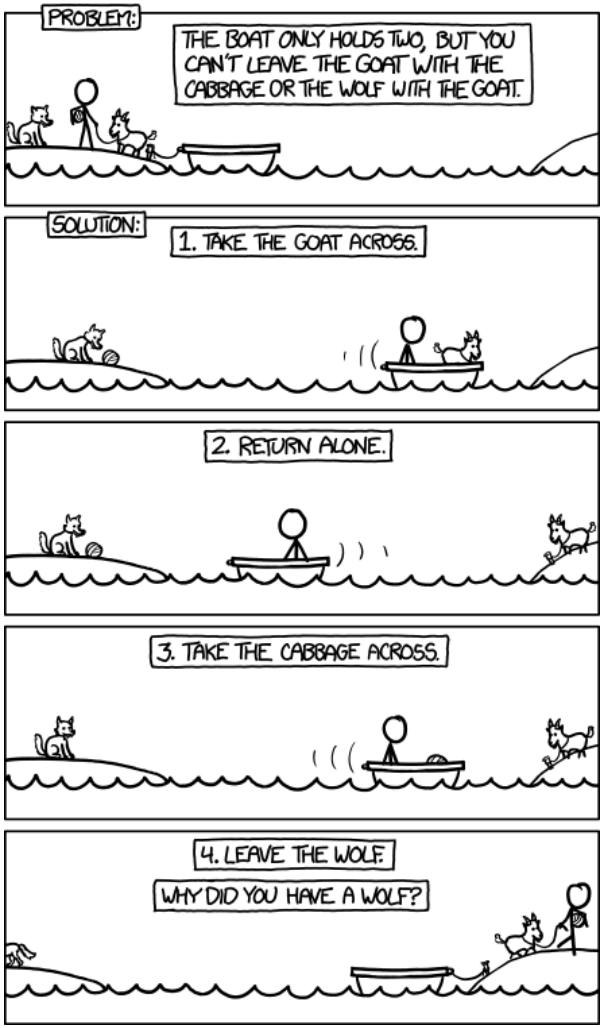
Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initial
- **Actions:**
  - Boat l
  - Load
  - Load goat, unload goat
  - Load cabbage, unload cabbage
  - (never leaving the wolf alone with goat or the goat alone with cabbage)
- **Goal:** (right, right, right, right)
- **Cost:** 1 per action

What's the size of the state space?

Example: Wolf, Goat, Cabbage



- **States:**
  - Location of wolf, goat, cabbage (left, boat, right) and boat (left, right)
  - Initia
- **Actions**
  - Boat
  - Load
  - Load
  - Load
  - (neve
    the g
- **Goal:** (r
- **Cost:** 1 per action

What's the size of the state space?

At most 3*3*3*2 = 54
(less when we apply the action constraints)

More Examples

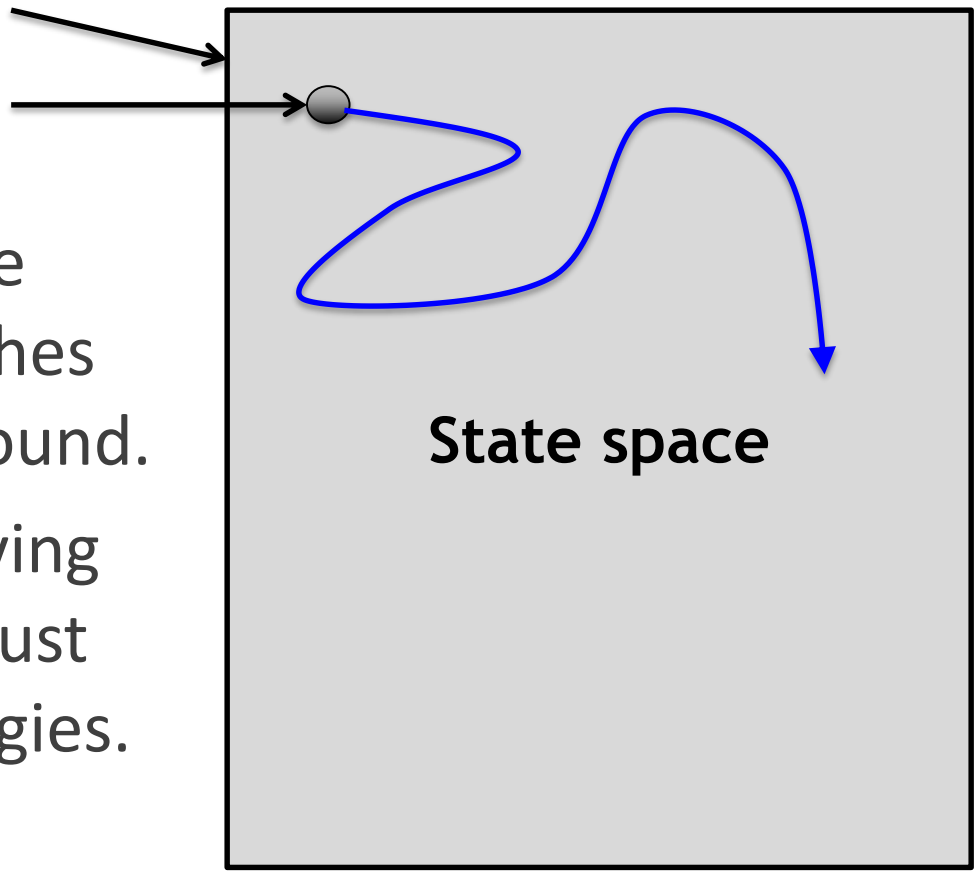
- Search Problems can be very general
- For example, we can formulate any of the following as a search problem:
  - Proving a theorem
  - Solving a physics problem
  - Finding your way around a city (path finding)
  - Playing individual sports (multiplayer sports and games belong to another category of problems)
  - Playing a computer game
  - “Where are my keys?”
  - Etc.

Solving Problems

- Let's only consider deterministic, fully-observable problems
- Simple algorithm:
  - Enumerate every possible sequence of actions:
    - [left]
    - [right]
    - [advance]
    - [left, left]
    - [left, right]
    - [left, advance]
    - …
  - Try one by one, until one reaches the solution

Solving Problems

- General Idea
  - Given the state space
  - Given the initial state
- Explore the state space until a state that matches the goal condition is found.
- Different problem-solving methods are actually just different search strategies.



Random Walk

- Algorithm:
  - Initial state: *S*
  - (1) Determine set of applicable actions *A* = validActions(*S*)
  - (2) Choose an action *a* in *A* at random
  - (3) Apply *a* to the current state: *S* = apply(*S*,*a*)
  - (4) If we have not reached the goal (*G*(*S*) == false), go to (1)
  - (5) Done
- Is this algorithm…
  - guaranteed to find a solution?
  - guaranteed to find the optimal solution? (i.e., shortest path or minimal cost)

Random Walk

- Algorithm:
  - Initial state: *S*
  - (1) Determine set of applicable actions *A* = validActions(*S*)
  - (2) Choose an action *a* in *A* at random
  - (3) Apply *a* to the current state: *S* = apply(*S*,*a*)
  - (4) If we have not reached the goal (*G*(*S*) == false), go to (1)
  - (5) Done
- Is this algorithm…
  - Is **not** guaranteed to find a solution?
  - Is **not** guaranteed to find the optimal solution? (i.e., shortest path or minimal cost)

Search

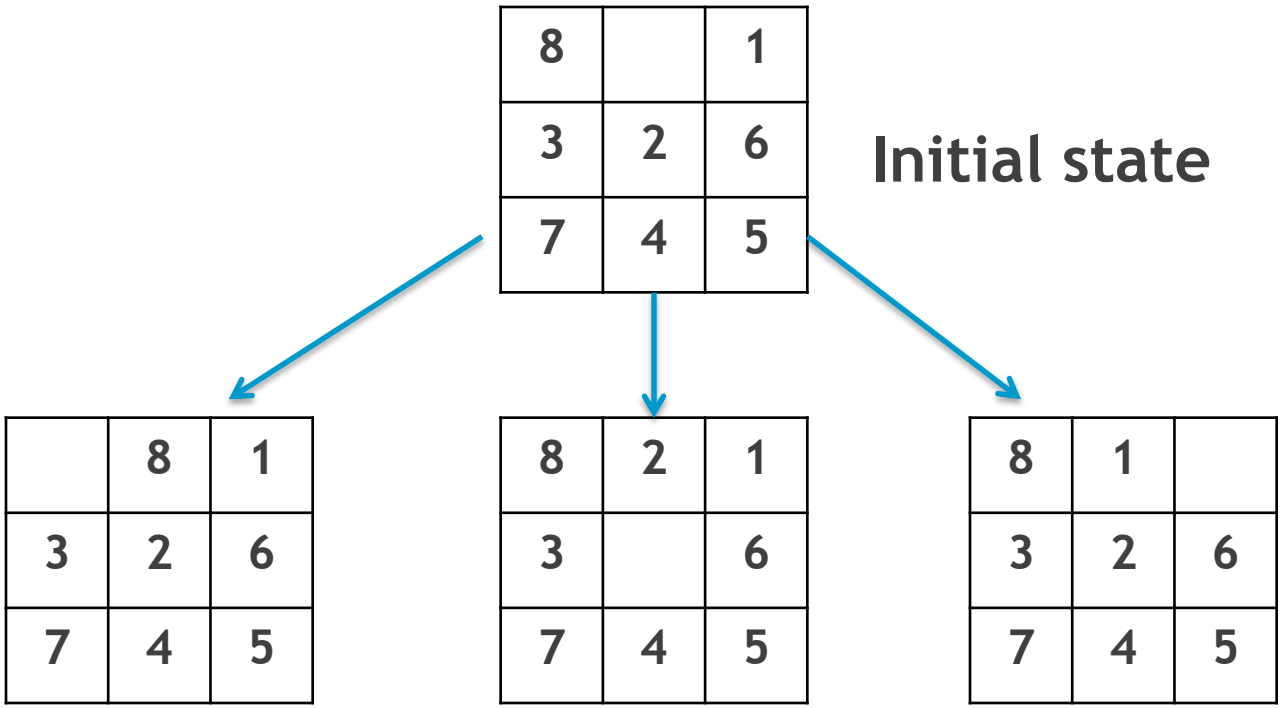
- Summary:
  - **Initial state**: initial configuration of the problem
  - **Actions**: set of actions the agent can perform to solve the problem
  - **State space**: set of possible states that are reachable from the initial space by any sequence of actions
- **Search**
  - Family of strategies to find solutions
  - Based on systematically exploring the state space until finding a sequence of actions that achieves the goal

Illustration

8		1
3	2	6
7	4	5

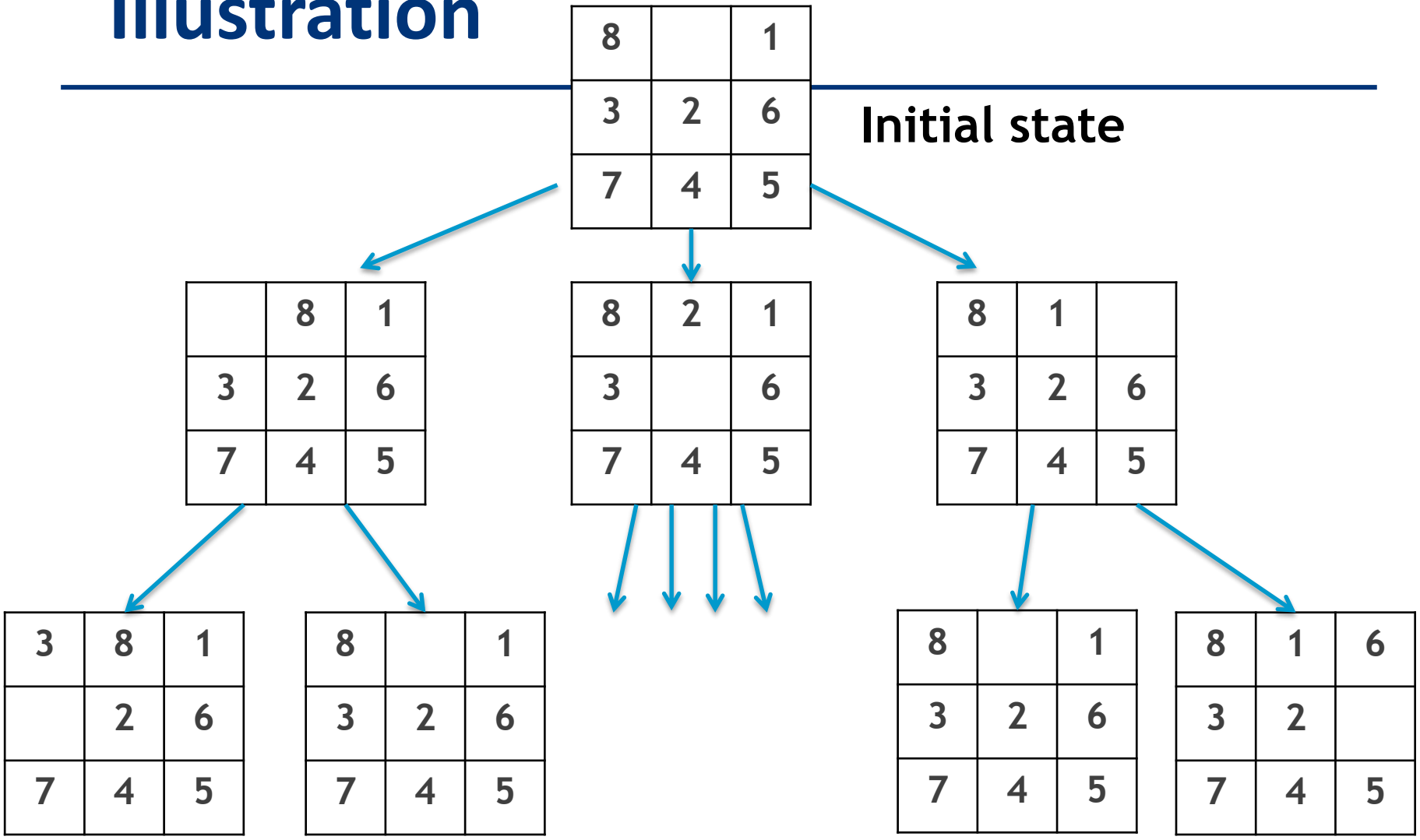
Initial state

Illustration



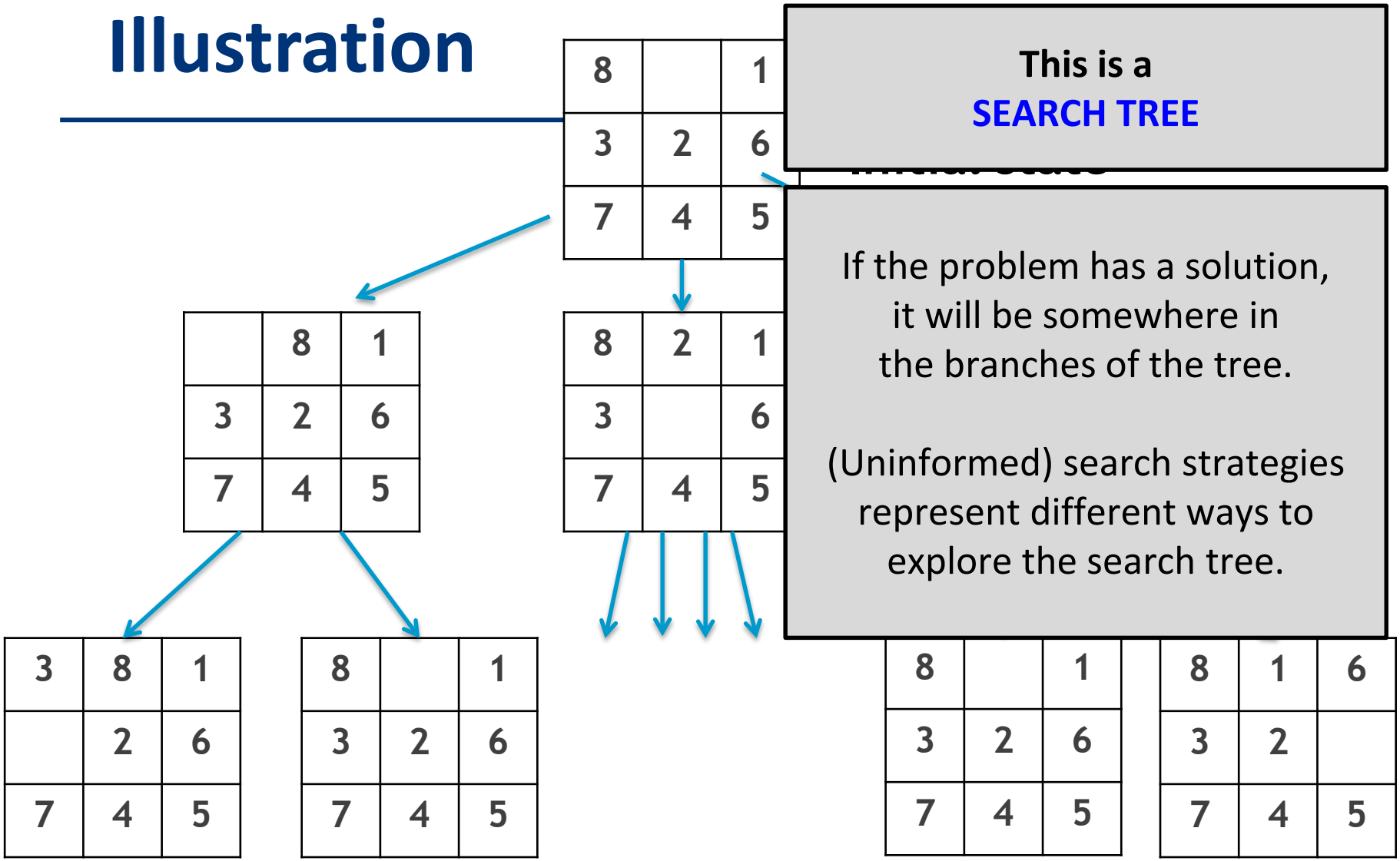
Set of states reachable by executing only one action

Illustration



Set of states reachable by executing two actions

Illustration

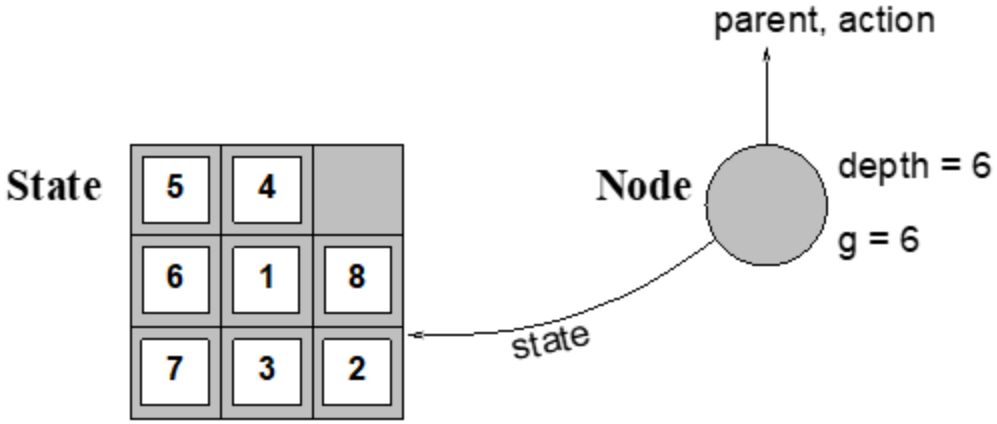


Set of states reachable by executing two actions

Implementation: states vs. nodes

- A state is a (representation of) a physical configuration
- A node is a data structure constituting part of a search tree that includes parent, children, depth, path cost g(x)

States do not have parents, children, depth, or path cost!



The Expand function creates new nodes, filling in the various fields and using the Successor function of the problem to create the corresponding states

Tree Search Algorithms

- Basic idea:
  - Offline, simulated exploration of the state space by generating successors of already-explored states (a.k.a. expanding states)

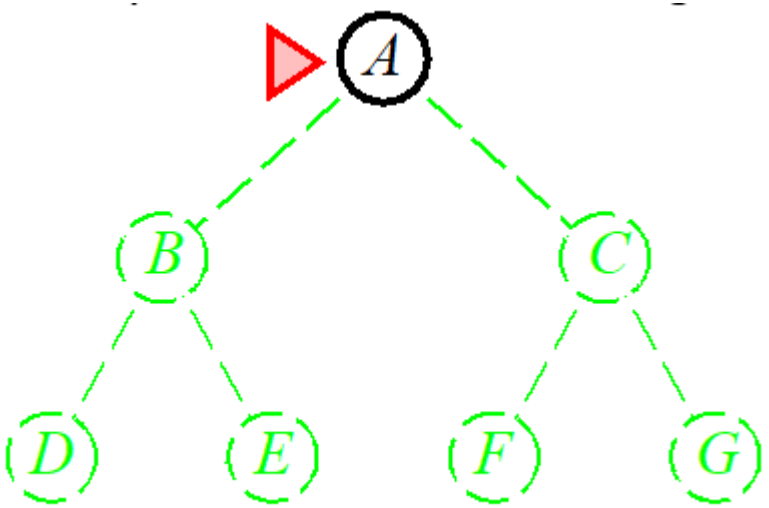
```
function Tree-Search(problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

Uninformed Search Strategies

- Uninformed strategies use only the information available in the problem definition
  - Breadth-first Search
  - Uniform-cost Search
  - Depth-first Search
  - Depth-limited Search
  - Iterative Deepening Search

Breadth-first Search

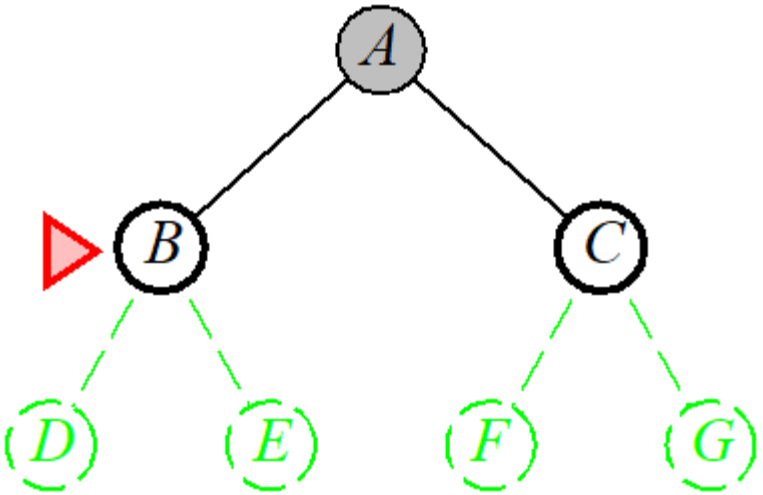
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



[A]

Breadth-first Search

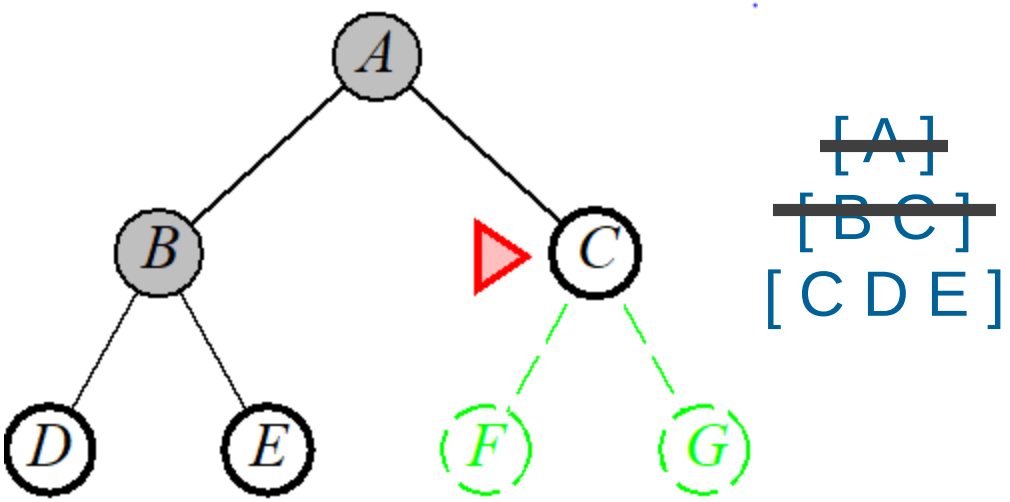
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



~~[A]~~
[B C]

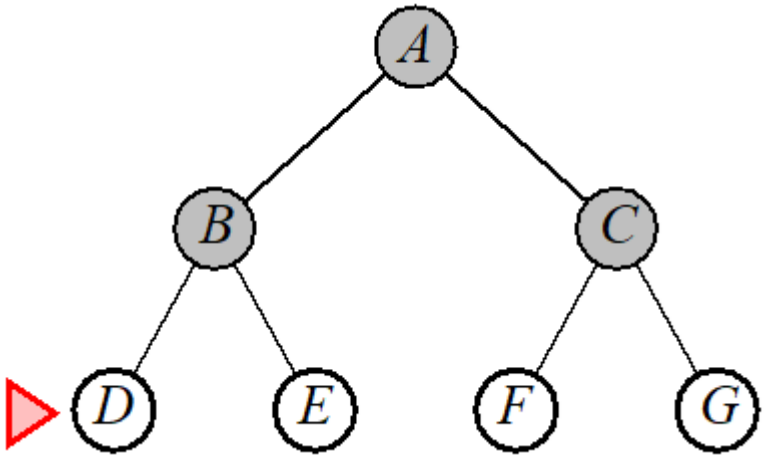
Breadth-first Search

- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



Breadth-first Search

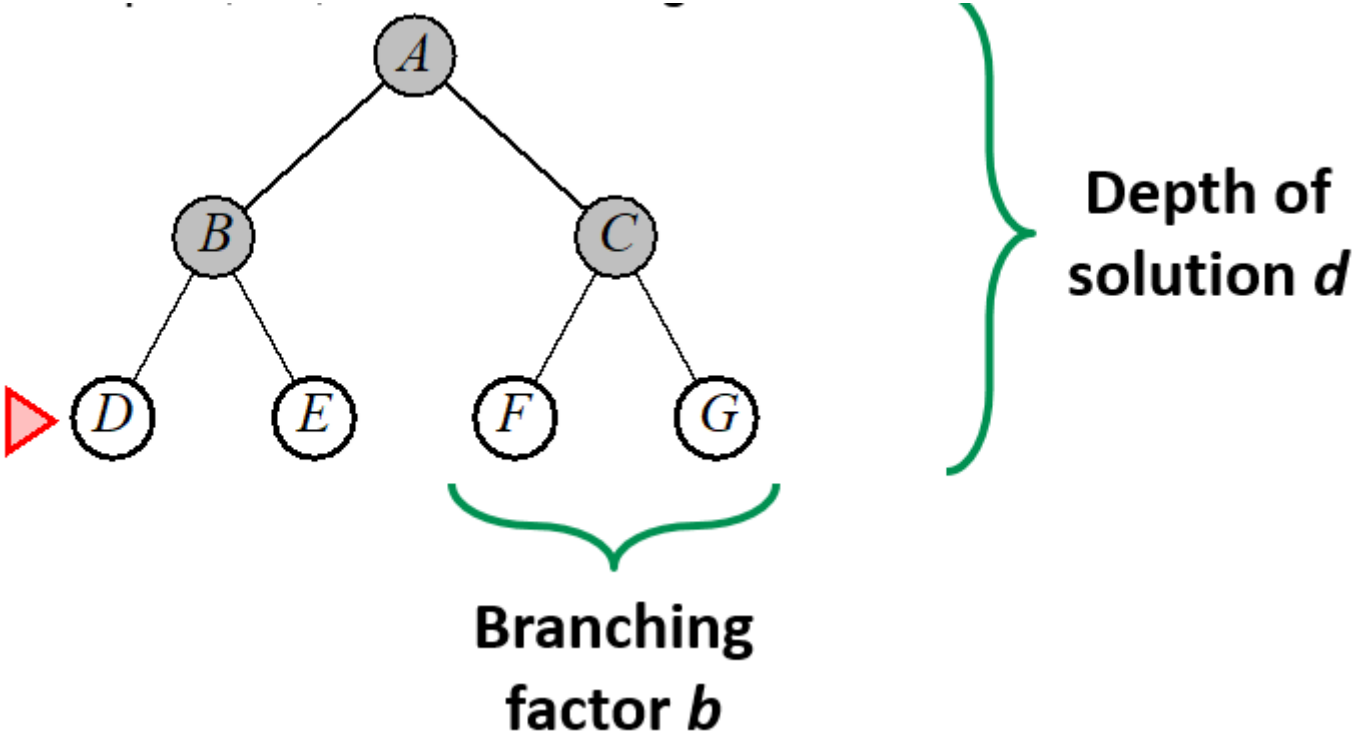
- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



~~[A]~~
~~[B C]~~
~~[C D E]~~
[D E F G]

Breadth-first Search

- Expand shallowest unexpanded node
- Implementation: *fringe* is a First In, First Out (FIFO) queue, i.e., new successors go at the end



Properties of Breadth-first Search

- Complete ?? – Will a solution always be found if one exists?
- Time ?? – How long will it take to find the solution? Often represented as the number of nodes searched.
- Space ?? – How much memory is needed to perform the search? Often represented as the maximum number of nodes stored at once.
- Optimal ?? – Will the optimal (least cost) solution be found?

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time ??
- Space ??
- Optimal ??

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space ??
- Optimal ??

Properties of Breadth-first Search

- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space – $O(b^d)$ (keeps every node in memory)
- Optimal ??

Properties of Breadth-first Search

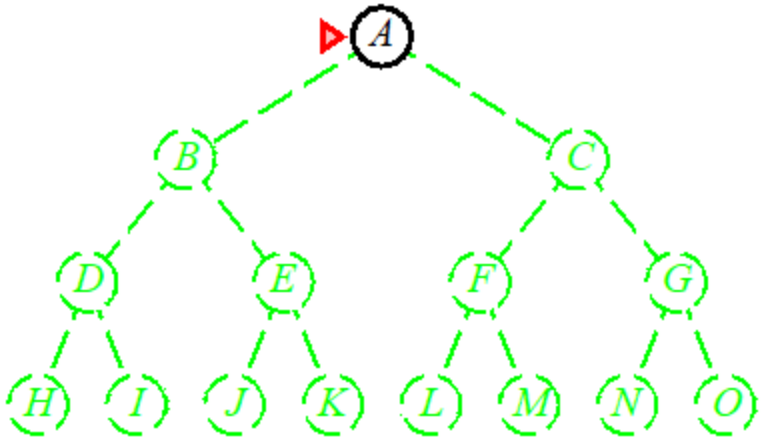
- Complete – Yes (if b is finite)
- Time – $1 + b + b^2 + b^3 + \ldots + b^d = O(b^d)$
  - i.e., exponential in d
- Space – O(b) (keeps every node in memory)
- Optimal – Yes (if cost = 1 per step); not optimal in general

Space is the big problem; can easily generate nodes at 100MB/sec so 24hrs = 8640GB.

Depth-first Search

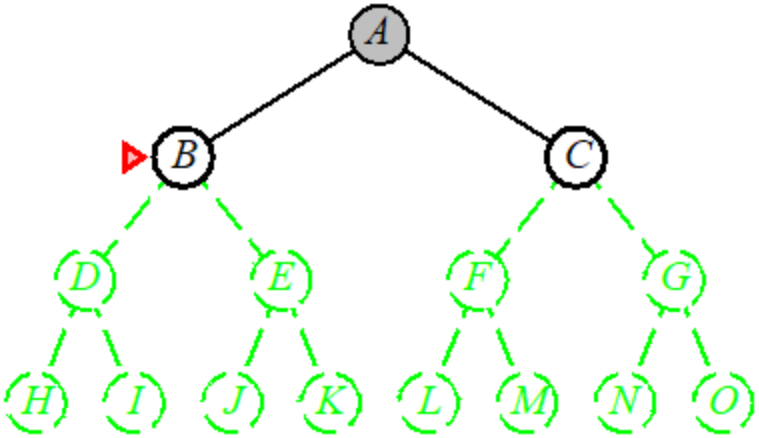
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

[A]



Depth-first Search

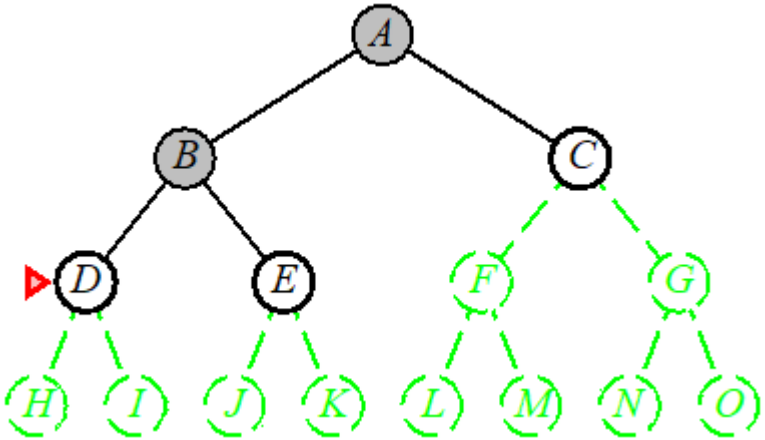
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
[B C]

Depth-first Search

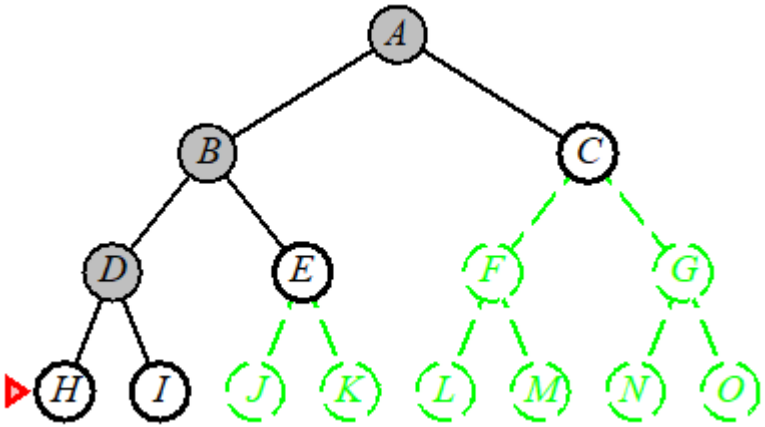
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
[D E C]

Depth-first Search

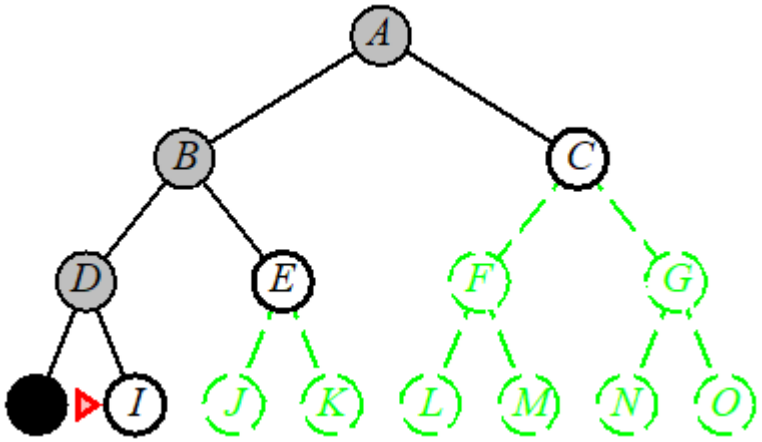
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
[H I E C]

Depth-first Search

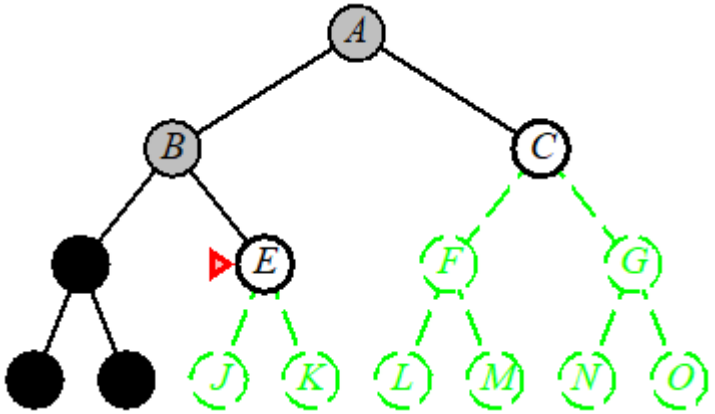
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
~~[H I E C]~~
[I E C]

Depth-first Search

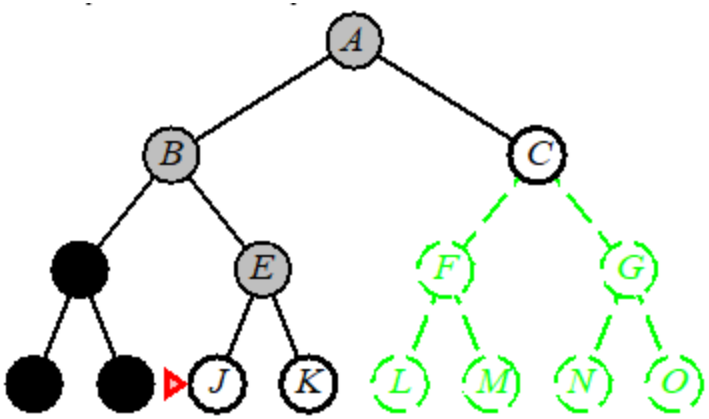
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



~~[A]~~
~~[B C]~~
~~[D E C]~~
~~[H I E C]~~
~~[I E C]~~
[E C]

Depth-first Search

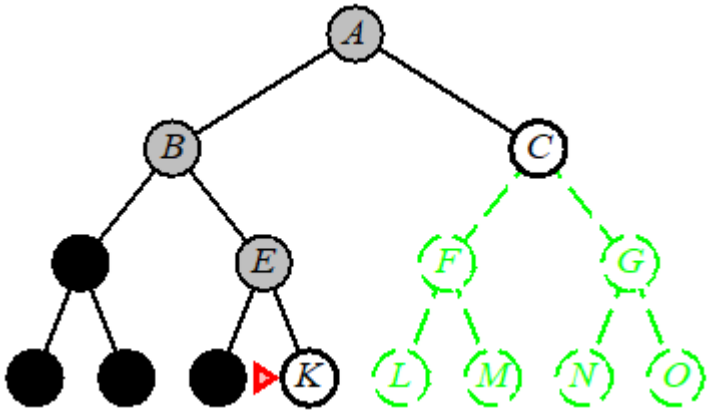
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front



[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]

Depth-first Search

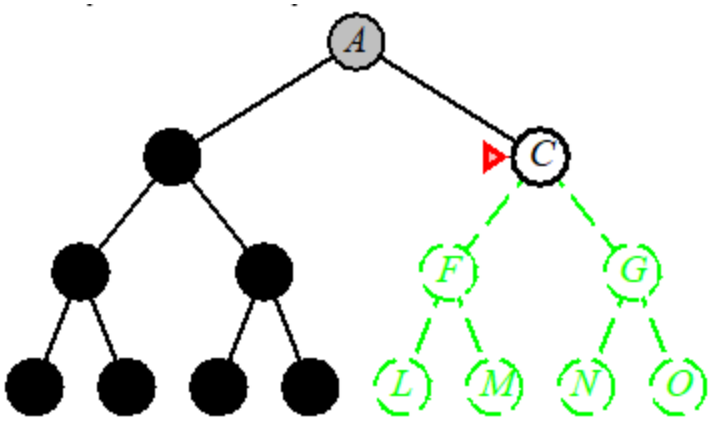
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

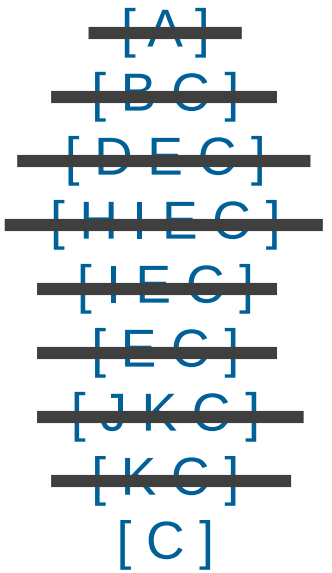


[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]
[K C]

Depth-first Search

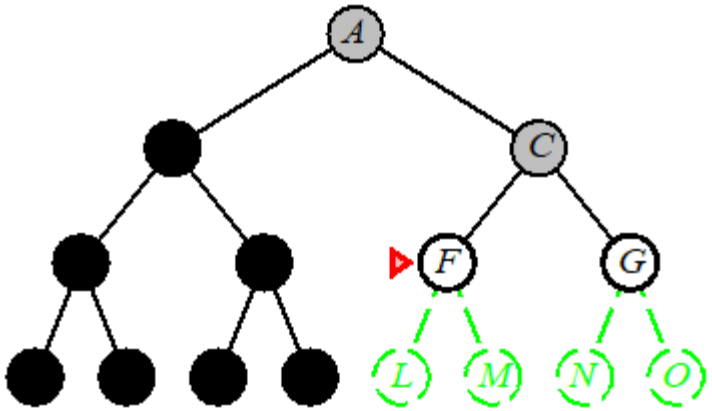
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

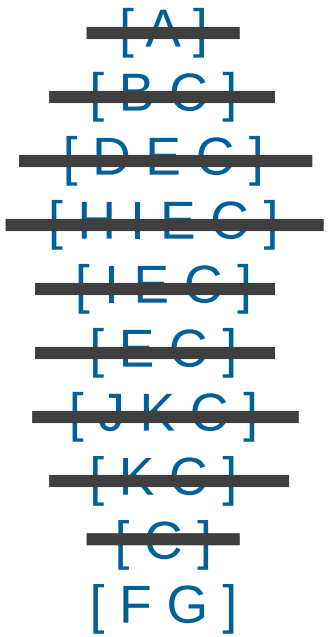




Depth-first Search

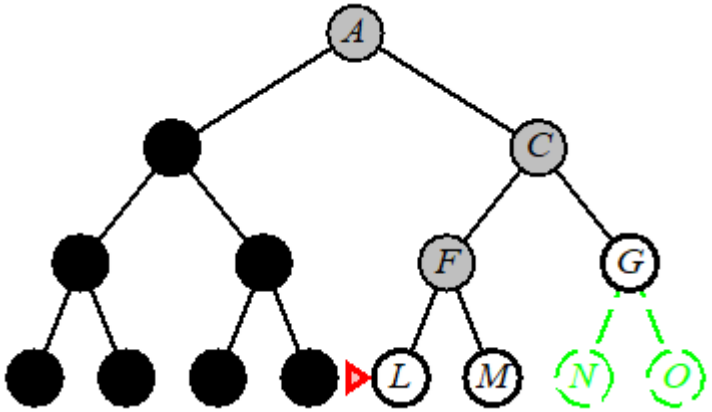
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front





Depth-first Search

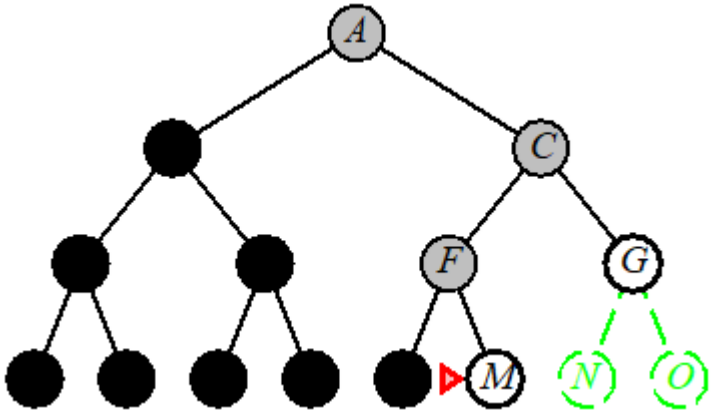
- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front

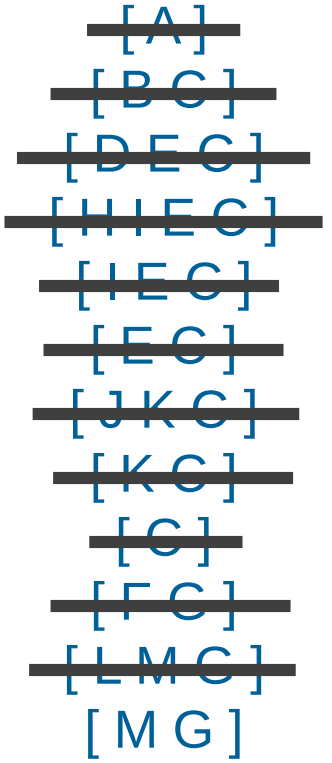


[A]
[B C]
[D E C]
[H I E C]
[I E C]
[E C]
[J K C]
[K C]
[C]
[F C]
[L M G]

Depth-first Search

- Expand deepest unexpanded node
- Implementation: *fringe* is a Last In, First Out (LIFO) queue, i.e., put successors at front





Properties of Depth-first Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time ??
- Space ??
- Optimal ??

Consider
m = maximum depth

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space ??
- Optimal ??

m = maximum depth;
bad if m >> d !

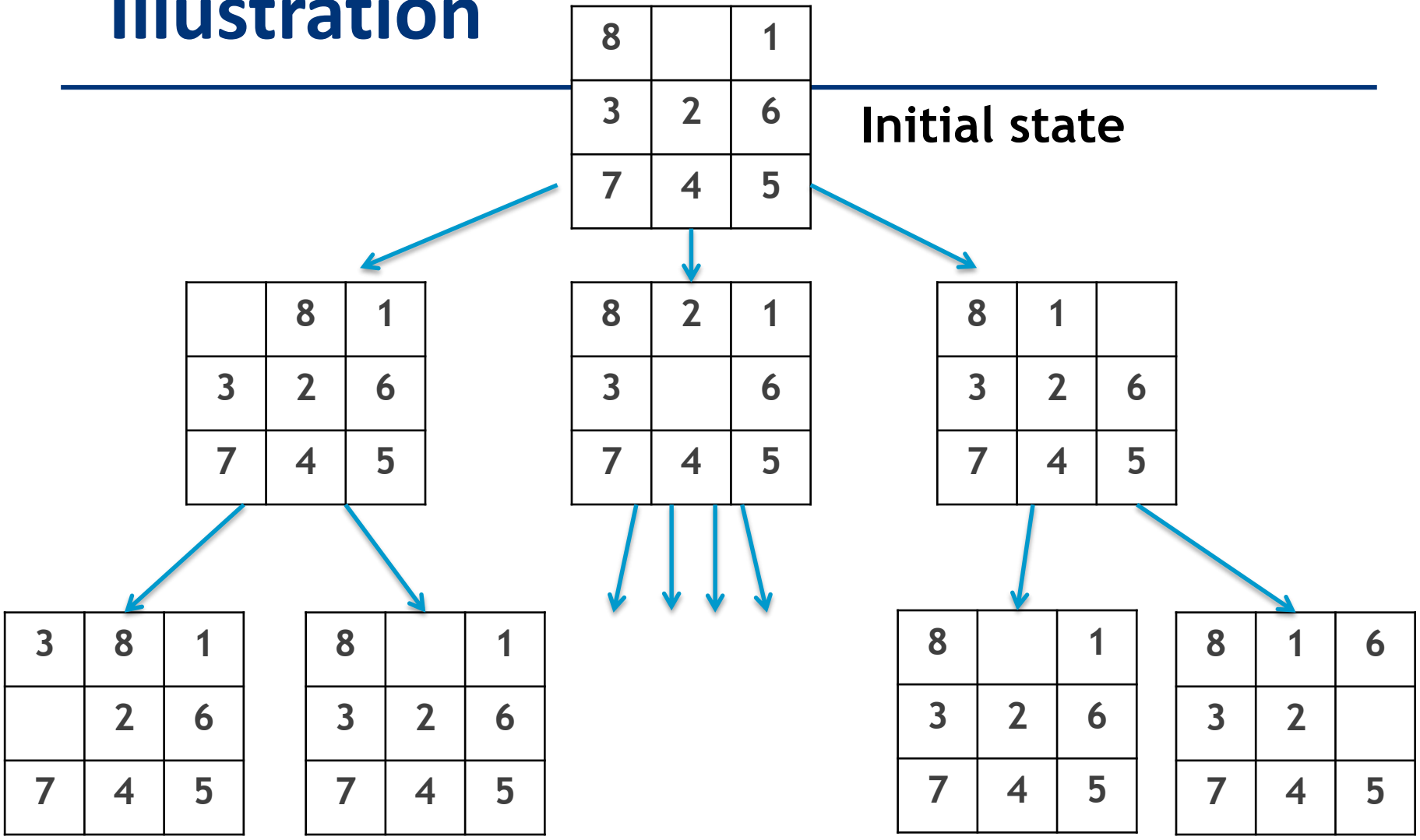
Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!
- Optimal ??

Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!
- Optimal – No

Illustration



Properties of Depth-first Search

- Complete - No: fails in infinite-depth spaces, spaces with loops. Modify to avoid repeated states along path -> complete in finite spaces
- Time – $O(b^m)$: terrible if m is much larger than d but if solutions are dense, may be much faster than breadth-first
- Space – O(bm), i.e., linear space!... But not really. In practice you need to store a list of already visited states to avoid infinite loops.
- Optimal – No

Depth-limited Search Algorithm

- Depth-first search with depth limit l, i.e., nodes at depth l have no successors

(Might be a good idea if *d* is well understood)

```
function Depth-Limited-Search(problem, limit) returns soln/fail/cutoff
  Recursive-DLS(Make-Node(Initial-State[problem]), problem, limit)

function Recursive-DLS(node, problem, limit) returns soln/fail/cutoff
  cutoff-occurred? ← false
  if Goal-Test(problem, State[node]) then return node
  else if Depth[node] = limit then return cutoff
  else for each successor in Expand(node, problem) do
      result ← Recursive-DLS(successor, problem, limit)
      if result = cutoff then cutoff-occurred? ← true
      else if result ≠ failure then return result
  if cutoff-occurred? then return cutoff else return failure
```

Iterative Deepening Search Algorithm

```
function Iterative-Deepening-Search(problem) returns a solution
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        result ← Depth-Limited-Search(problem, depth)
        if result ≠ cutoff then return result
    end
```

Iterative Deepening Search l = 0

Limit = 0

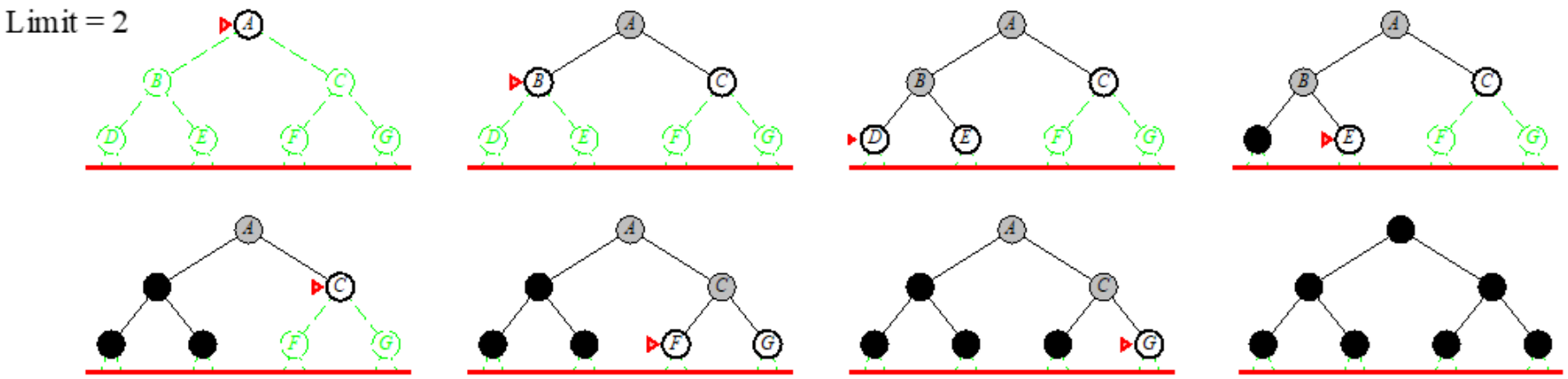




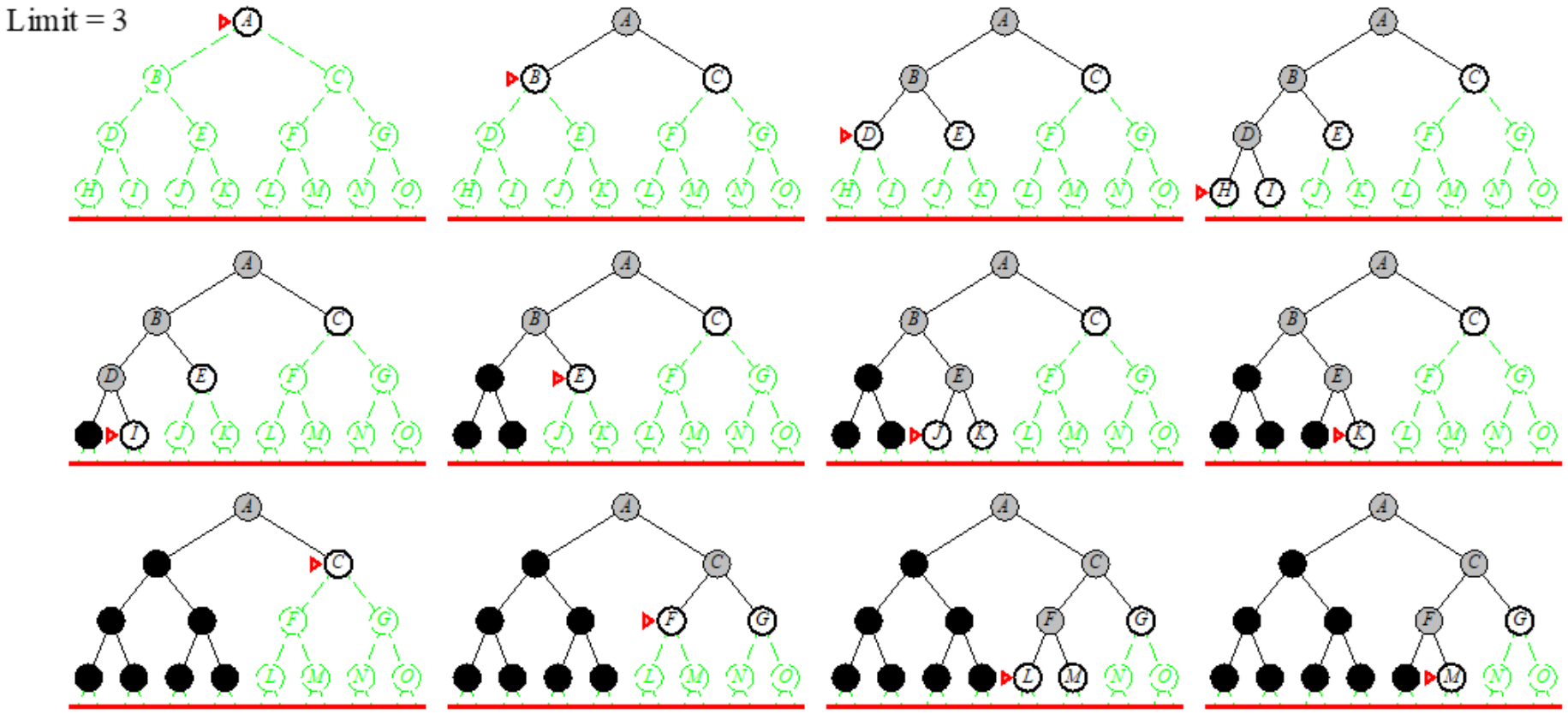
Iterative Deepening Search l = 1



Iterative Deepening Search l = 2



Iterative Deepening Search l = 3



Properties of Iterative Deepening Search

- Complete ??
- Time ??
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time ??
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
  - Like BFS
- Space ??
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
  - Like BFS
- Space – O(bd)
  - Similar to DFS
- Optimal ??

Properties of Iterative Deepening Search

- Complete – Yes
- Time – $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- Space – O(bd)
- Optimal – Yes if step cost = 1;
  - Numerical comparison for b = 10 and d = 5, solution at far right leaf:

    N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450

    N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 + 999,990 = 1,111,100
  - IDS does better because other nodes at depth d are not expanded
  - BFS can be modified to apply goal test when a node is generated

Summary of Algorithms

Criterion	Breadth-First	Depth-First	Depth-Limited	Iterative Deepening
Complete ?	Yes*	No	Yes, if $l \geq d$	Yes
Time ?	b^d	b^m	b^l	b^d
Space ?	b^d	bm	bl	bd
Optimal ?	Yes*	No	No	Yes*

Coming up…

- Informed search strategies